KAREMOPS.AI · FIELD GUIDE

Build Your First Claude Code Skill.

From zero to a working slash command. The folder, the file, the trigger description, and one full example you can ship today.

Inside

- Why one-off prompts are the trap (and skills are the move)
- The 3 questions every skill answers
- The folder structure + SKILL.md anatomy
- How to write a trigger description Claude actually fires on
- One full example built end-to-end: `/daily-standup`

Karem AlMoayyad
KAREMOPS.AI · ARC DIGITAL

@KAREMOPS.AI
IG · TIKTOK · LINKEDIN

§ 01 · THE MINDSET SHIFT

Prompts are disposable. Skills are leverage.

Most people open Claude Code and start typing the same prompts they typed yesterday. Different wording, same idea. That's the trap.

Basically, every time you re-explain context to Claude, you're paying a tax. The tax is your time, your tokens, and the fact that the result drifts a little every time because you phrased it slightly different.

A skill solves all three. You write the instructions *once*, in a file Claude reads every time you trigger it. Same context, same structure, same output quality. The only thing that changes is the input.

One-off prompts vs skills

PROMPT	SKILL
Typed fresh each time. Lives in your head. Drifts. Forgets your context. Dies when you close the chat.	Written once. Lives in a file. Repeatable. Carries your context. Available every session, every project.

> **The shift:** Stop asking "what should I type next?" Start asking "what process do I keep repeating that should live in a file?"

The 3 questions every skill answers.

Before you write a single line of a skill, answer these three. Skip this and your skill either never fires, fires at the wrong time, or fires and does the wrong thing.

1. **Trigger** — when should this fire? What words, what task, what situation? This is what Claude reads to decide whether to use your skill at all.
2. **Input** — what does the skill need to do its job? A URL, a file path, a topic, raw text, a record ID? Be specific.
3. **Output** — what does done look like? A file written to disk, a record updated in Airtable, a draft in your editor, a message in Slack?

Example: `/daily-standup`

- **Trigger:** User asks for a standup update, wants to know what they did yesterday, or types `/daily-standup`
- **Input:** Yesterday's calendar events + yesterday's git commits across the user's active repos
- **Output:** A 3-bullet standup in the format: "Yesterday / Today / Blockers"

> **The test:** If you can't write those three lines in one sentence each, you don't have a skill yet. You have a vibe. Write the lines first, file second.

STEP 01

The folder and the SKILL.md file.

A skill is one folder with one file in it. That's the entire structure.

The folder

Claude Code looks for skills in two places — your global skills folder and your project's local skills folder.

```
~/.claude/skills/your-skill-name/       ← available in every project
.claude/skills/your-skill-name/         ← project-specific only
```

Use global for skills you'll reuse across every project (your content workflow, your standup, your weekly review). Use project-local for skills tied to one codebase or client.

The file

Inside that folder, create one file: `SKILL.md`. That's where everything lives — the trigger description, the instructions, the references.

```
~/.claude/skills/daily-standup/
└── SKILL.md
```

> **That's it.** No package.json, no config, no install command. One folder, one markdown file. The whole system is files Claude reads.

STEP 02

The trigger description is the whole game.

If your skill never fires, this is why. Claude reads the description and decides whether your skill is relevant to what the user just asked. Vague description = Claude ignores it.

Anatomy of the SKILL.md frontmatter

```
---
name: daily-standup
description: [THIS IS THE WHOLE GAME]
---
```

Two fields. The `name` becomes your slash command (`/daily-standup`). The `description` is what Claude scans to decide if your skill applies.

Bad vs good descriptions

WON'T FIRE

"Helps with standup updates."

No trigger words. No input. No output. Claude has nothing to match against.

FIRES RELIABLY

"Generate a daily standup update by scanning today's calendar events and yesterday's git commits. Use when the user asks for a standup, asks what they did yesterday, or needs a status update for their team."

Names the output. Names the inputs. Names the trigger situations.

> **The formula:** [What it does] by [how it does it]. Use when [trigger situation 1], [trigger situation 2], or [trigger situation 3].

STEP 03

The body — instructions Claude follows.

Everything under the frontmatter is the actual playbook. Treat it like you're onboarding a new employee on day one — explicit, sequenced, no assumptions.

Here's the full `SKILL.md` for `/daily-standup`, end-to-end:

```
---
name: daily-standup
description: Generate a daily standup update by scanning today's calendar events and
yesterday's git commits. Use when the user asks for a standup, asks what they did
yesterday, or needs a status update for their team.
---

# Daily Standup

Generate a 3-bullet standup update in the "Yesterday / Today / Blockers" format.

## Steps

1. Run `git log --since="yesterday" --until="today" --author="$(git config user.email)"
--oneline` in the user's active repos. Pull the commit messages.

2. Read today's calendar events (use the calendar MCP if connected, otherwise ask the
user to paste them).

3. Summarize yesterday's commits into 1-2 bullets — what shipped, what's in progress.
Skip noise commits (typos, formatting).

4. Pull today's calendar to list what the user is heads-down on.

5. Ask the user one question: any blockers? If yes, add as third bullet. If no, write
"None."

## Output format

**Yesterday:** [1-2 bullets of what shipped]
**Today:** [1-2 bullets from the calendar — meetings + focus blocks]
**Blockers:** [user's answer, or "None"]

Copy the final output to clipboard.
```

Make sure it actually fires.

A skill that lives in a folder but never triggers is just a markdown file. Test it the second you save it.

The 30-second test

1. Open Claude Code in any project.
2. Type `/daily-standup` — if the slash menu autocompletes your skill name, the file is being read.
3. Hit enter. Watch what Claude does. Does it follow your steps? Does it ask for the right inputs? Does the output match your format?
4. If yes — ship it. If no — open the SKILL.md, fix the gap, save, rerun.

Common reasons it won't fire

- **Description too vague.** "Helps with X" is invisible to Claude. Rewrite with the formula on page 4.
- **No trigger phrases.** Add 2-3 specific situations: "Use when the user asks for X, mentions Y, or types Z."
- **Wrong folder.** Global goes in `~/.claude/skills/`, not `~/claude/skills/`. The dot matters.
- **Description overlaps with another skill.** Claude picks one. If two skills compete, the more specific one wins. Tighten yours.

> **The loop:** Write, test, watch it fail or succeed, rewrite the description, retest. Most skills take 3-4 iterations on the description alone before they fire reliably. That's normal — not a sign you're doing it wrong.

§ PAYOFF

You just built one skill.

That's the whole system. Folder → SKILL.md → trigger description → body → test. Do that 10 more times for the tasks you actually repeat every week and you've replaced half your workflow.

The reason the people you're watching on Instagram look like they're moving 10x faster is not because they're prompting harder. They built skills. The prompts live in files now.

What's next

You just felt how much work one skill is. The description rewrites. The format tweaks. The 4 iterations before it fires clean.

Now imagine four of them, already wired together, that take an idea and turn it into 21 ideated, scripted, designed, and posted videos. With their own analysis loop that makes them sharper every week from the data they generate.

The Karemops Skill Library — inside Skool

- `/content-ideator` — pulls topics from performance data + competitor wins
- `/content-scripter` — writes hooks and full filming cards
- `/carousel-creator` — generates slides end-to-end
- `/post-content` — schedules across IG, TikTok, LinkedIn

Plus the analysis skills, the references, the chaining setup. Install and run — skip the 3 months of iteration.

> **If this hit, send it to one other person who's still typing the same prompts every day.**